

Sound Notional Machines

ANONYMOUS AUTHOR(S)

A notional machine is a pedagogical device that abstracts away details of the semantics of a programming language to focus on some aspects of interest. A notional machine should be *sound*: it should be consistent with the corresponding programming language, and it should be a proper abstraction. This reduces the risk of it introducing misconceptions in education. Despite being widely used in computer science education, notional machines are usually not evaluated with respect to their soundness. To address this problem, we first introduce a formal definition of soundness for notional machines. The definition is based on the construction of a commutative diagram that relates the notional machine with the aspect of the programming language under its focus. Derived from this formalism, we present a methodology for constructing sound notional machines, which we demonstrate by applying it to a series of small case studies. We also show how the same formalism can be used to analyze existing notional machines and find inconsistencies in them as well as propose solutions to these inconsistencies. The work establishes a firmer ground for research in notional machines by serving as a framework to reason about them.

CCS Concepts: • **Social and professional topics** → **Computing education**; • **Software and its engineering** → **Formal language definitions**; **Context specific languages**.

Additional Key Words and Phrases: notional machines, programming education, language semantics, equational reasoning, bisimulation

ACM Reference Format:

Anonymous Author(s). 2018. Sound Notional Machines. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 30 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Learning to program involves learning how to express a program in a programming language, but also learning what the semantics of such a program is. For novices, the semantics of a program is often not obviously apparent from the program itself. Instructors then often use a *notional machine* [19] to help teach some particular aspect of programs and programming languages, and also to assess students' understanding of said aspect. This aspect is the notional machine's focus. For example, showing expressions as trees (the EXP_{TREE} notional machine), as depicted in Figure 1, brings out the internal structure of lambda-calculus expressions [26] and can help to explain the step-by-step evaluation of such expressions. Notional machines are used widely in computer science education: Fincher et al. [2020] interviewed computer science teachers to build up a dataset of 37 notional machines¹.

1.1 Unsound Notional Machines

Given the extensive use of notional machines, and their intended use as devices to help students when learning, it is important to look at their quality. To begin with, one should make sure that the notional machine is *sound*: it is faithful to the aspect of programs it is meant to focus on. Anecdotal evidence of using unsound representations in education goes

¹<https://notionalmachines.github.io/notional-machines.html> - The website lists 57 items but some of them are part of what they call a notional machine sequence, which we consider a single notional machine.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

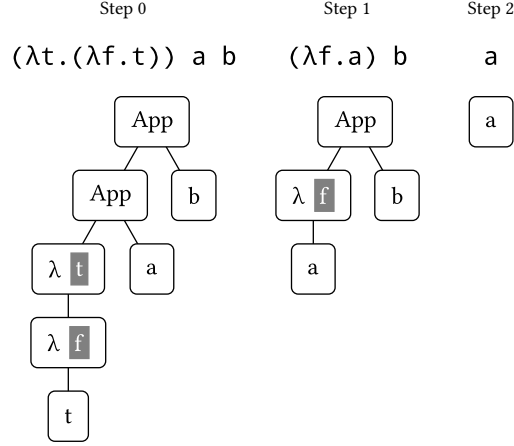


Fig. 1. Evaluation of the term $(\lambda t.(\lambda f.t)) a b$ shown with the notional machine ExpTREE.

back a long way. In 1960, education pioneer Jerome Bruner wrote that “the task of teaching a subject to a child at any particular age is one of representing the structure of that subject in terms of the child’s way of viewing things”, that this “task can be thought of as one of translation”, but that ideas have to be “represented *honestly* [...] in the thought forms of children” [6]. Bruner’s use of the term “honestly” can be seen as a call for soundness of such representations. Decades later, Richard Feynman eloquently stated [18], after reviewing “seventeen feet” of new mathematics schoolbooks for the California State Curriculum Commission:

[The books] would try to be rigorous, but they would use examples (like automobiles in the street for “sets”) which were almost OK, but in which there were always some subtleties. The definitions weren’t accurate. Everything was a little bit ambiguous.

Ambiguously specified notional machines and notional machines with imperfect analogies to programming concepts are a problem. Educators may mischaracterize language features and students may end up with misconceptions [7] instead of profoundly understanding the language.

For example, Fincher et al. [19] describe the “Array as Row of Spaces in Parking Lot” notional machine. Figure 2 shows part of their card summarizing it. Notice the parallels between the programming language (PL) and the notional machine (NM). Consider Java, a language commonly used in programming courses. In Java, when an array of objects is allocated, all its slots contain **null**, which means these slots don’t contain a reference to any object. This would be reasonably represented in the notional machine as an empty parking lot. But if instead of an array of objects, we have an array of **ints**, for example, then when we instantiate an array, all its slots contain $\emptyset$, which is not the absence of a number but a number like any other. A student could also reasonably question whether one can park a car in a slot that is already occupied by another car, or whether one has to remove a car from a spot to park another car in the same spot. In fact, the authors point out that, “the effectiveness of the analogy depends on [...] how well that models the semantics of the programming language.”

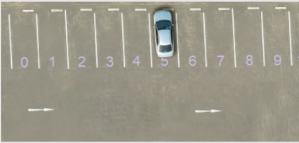
Array as Row of Spaces in Parking Lot	
Programming Paradigm: imperative Programming Language: any	
	
Conceptual Advantage Builds from the notion of a variable as a parking space and leverages the notion that spaces in larger parking lots are often numbered.	
PL	NM
array	row of cars in a parking lot
array index	space number in lot
array element	car
array value	specific car in specific space
Form Analogy	
Draws Attention To The use of indices in arrays as well as the array's construction from contiguous adjacent variables in memory.	
Cost Short time to introduce but learners need to be familiar with parking lots and how someone might locate a specific car in a row in a parking lot.	
Notes/Other If in a statically-typed language one can discuss rows that are for cars only, trucks only, motorcycles only, etc.	
Origin/Source Own practice	
Attribution Used by Jan, collected by <i>Jan</i>	

Fig. 2. The “Array as Row of Spaces in Parking Lot” notional machine captured by Fincher et al. [19].

1.2 Soundness of Notional Machines via Simulation

To avoid these issues, we need to make sure that a notional machine is indeed an accurate abstraction. Although the more general view of notional machines as “pedagogic devices to assist the understanding of some aspect of programs or programming” [19] makes no direct reference to programming languages, programs are expressed in programming languages so we will look at these “aspects of programs” through the lens of how they are realized by some programming language. If a notional machine represents a part of the operational semantics of a programming language, for example, then this representation should be sound, in the sense that steps in the notional machine correspond to steps in the operational semantics of the programming language.

Showing the soundness of a notional machine amounts to demonstrating that the notional machine *simulates* (in the sense described by Milner [27]) the aspect of programs under the focus of the notional machine. This property can be given in the form of a commutative diagram. Milner’s simulation was also used by Hoare [22] to establish a definition of the correctness of an ‘abstract’ data type representation with respect to its corresponding ‘concrete’ data

type representation. Cousot and Cousot [11] use a similar commutative diagram to simulate concrete computations in their abstract interpretation framework. This interpretation of simulation also captures the relationship between a notional machine and the underlying programming language because a notional machine is indeed an *abstraction* of some aspect of interest.

1.3 Contributions

This paper makes the following contributions:

- A formal definition of sound notional machine based on simulation (Section 2.1.2).
- A methodology for designing notional machines that are sound by construction.
It consists of deriving part of the notional machine by leveraging the relationship between the abstract representation of the notional machine and the abstract syntax of the programming language under its focus.
We demonstrate the methodology by applying it to a combination of various small programming languages, notional machines, and aspects of programming language semantics (Section 2).
- A methodology for analyzing notional machines with respect to their soundness.
It consists of modelling the notional machine and the aspect of the programming language under its focus, relating them via simulation.
We demonstrate the methodology by analyzing existing notional machines, sometimes pointing out unsoundnesses, and suggesting directions for improvement (Section 3).

A brief discussion about the distinction between the abstract and concrete representations of a notional machine follows in Section 4. We then evaluate, in Section 5, the entire framework by comparing the notional machines that appeared in Section 2 and Section 3 to an existing dataset of 37 notional machines. We classify the notional machines in the dataset according to various dimensions and show that the notional machines we analyzed are representative of the design space of notional machines used in practice. Section 6 discusses related work and Section 7 concludes.

2 Designing Sound Notional Machines

We can only begin to talk about the soundness of a notional machine if we have a formal description of the programming language the notional machine is focused on. We use a set of small programming languages with well-known formalizations described in Pierce’s Types and Programming Languages (TAPL) book [28]. The languages are used to explore different aspects of programming language semantics. Table 1 shows the notional machines we use in this section as well as the corresponding programming language and aspect of the semantics of the programming language that the notional machine focuses on. We use the first example also to introduce the definition of soundness for notional machines.

Section	Notional Machine	Programming Language	Focus
2.1	EXPTREE	UNTYPEDLAMBDA	Evaluation
2.2	EXPTUTORDIAGRAM	UNTYPEDLAMBDA	Evaluation
2.3	TAPLMEMORYDIAGRAM	TYPEDLAMBDAREF	Evaluation (Refs)
2.4	TYPEDEXPTUTORDIAGRAM	TYPEDARITH	Type-checking

Table 1. Notional machines, programming languages, and aspects of focus used in Section 2.

$$\begin{array}{ccc}
 A_{NM} & \xrightarrow{f_{NM}} & B_{NM} \\
 \uparrow \alpha_A & & \uparrow \alpha_B \\
 A_{PL} & \xrightarrow{f_{PL}} & B_{PL}
 \end{array}$$

$$\alpha_B \circ f_{PL} \equiv f_{NM} \circ \alpha_A \quad (1)$$

Fig. 3. Soundness condition for notional machines shown as a commutative diagram and in algebraic form.

We model each programming language and notional machine in Haskell. The models are executable, so they include implementations of the programming languages (including parsers, interpreters, and type-checkers), the notional machines, and the relationship between them². The soundness proofs presented in this section are done using equational reasoning [4, 20].

2.1 Isomorphic Notional Machines

As a first straightforward example, let's look at a notional machine for teaching how evaluation works in the untyped lambda-calculus (we will refer to this language as `UNTYPEDLAMBDA`³). While most research papers discuss the lambda-calculus using its textual representation, textbooks sometimes illustrate it using tree diagrams [28, p. 54]. We use this as an opportunity to define a simple notional machine which we call `EXPTREE`.

2.1.1 Illustrative Example. In Figure 1, `EXPTREE` is used to demonstrate the evaluation of a specific lambda expression, which happens in two reduction steps. The top of the figure shows the terms in the traditional textual representation of the programming language, while the bottom shows the terms as a tree.

2.1.2 Soundness via Commutative Diagrams. In general, a notional machine is sound if the diagram in Figure 3 commutes. We call the commutativity of this diagram the *soundness condition* for a notional machine. In this diagram, the vertices are types and the edges are functions. We will explain the diagram while instantiating it for this illustrative example (the result of this instantiation is shown in Figure 4).

The bottom layer (A_{PL}, f_{PL}, B_{PL}) represents the aspect of a programming language⁴ we want to focus on. A_{PL} is an abstract representation of a program in that language. In our example, that is the abstract syntax of `UNTYPEDLAMBDA` (given by the type $Term_{U\lambda}$). The function f_{PL} is an operation the notional machine is focusing on. In our example, that would be *step*, a function that performs a reduction step in the evaluation of a program according to the operational semantics of the language, which in this case also produces a value of type $Term_{U\lambda}$.

The top layer of the diagram (A_{NM}, f_{NM}, B_{NM}) represents the notional machine. A_{NM} is an abstract representation of the notional machine (its abstract syntax). In our simple example, that is a type $ExpTree$ trivially isomorphic to $Term_{U\lambda}$ via a simple renaming of constructors. The function f_{NM} is an operation on the notional machine which should

²The artifact containing a superset of the examples in this paper is at [10.5281/zenodo.12609636](https://zenodo.org/record/12609636).

³The syntax and reduction rules for `UNTYPEDLAMBDA` are reproduced in the appendix provided as supplementary material.

⁴Although we refer to the bottom layer of the diagram as the programming language layer and we restrict ourselves to analyzing aspects of the syntax and semantics of programming languages, for which we have well-established formalizations, that is not an intrinsic restriction of the approach. In principle, the bottom level of the diagram can be whatever aspects of programs or programming the notional machine is focused on.

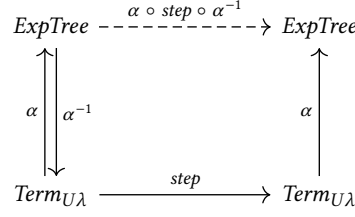


Fig. 4. Instantiation of the commutative diagram in Figure 3 for the notional machine `ExpTree` and the programming language `UNTYPEDLAMBDA`.

correspond to f_{PL} . Connecting the bottom layer to the top layer, there are the functions α_A and α_B from the abstract representation of a program in the programming language to the abstract representation of the notional machine. α is also called an abstraction function.

Definition 2.1. Given the notional machine $(A_{NM}, B_{NM}, f_{NM} :: A_{NM} \rightarrow B_{NM})$, focused on the aspect of a programming language given by $(A_{PL}, B_{PL}, f_{PL} :: A_{PL} \rightarrow B_{PL})$, the notional machine is *sound* iff there exist two functions $\alpha_A :: A_{PL} \rightarrow A_{NM}$ and $\alpha_B :: B_{PL} \rightarrow B_{NM}$ such that $\alpha_B \circ f_{PL} \equiv f_{NM} \circ \alpha_A$.

If the abstract representation of the programming language (A_{PL}) is isomorphic to the abstract representation of the notional machine (A_{NM}) , we can construct an inverse mapping α_A^{-1} such that $\alpha_A^{-1} \circ \alpha_A \equiv id \equiv \alpha_A \circ \alpha_A^{-1}$. In that case, we can always define a correct-by-construction operation f_{NM} on A_{NM} in terms of an operation f_{PL} on A_{PL} :

$$\begin{aligned} f_{NM} &:: A_{NM} \rightarrow B_{NM} \\ f_{NM} &= \alpha_B \circ f_{PL} \circ \alpha_A^{-1} \end{aligned}$$

In such cases, the diagram always commutes and therefore the notional machine is sound:

$$f_{NM} \circ \alpha_A \equiv \alpha_B \circ f_{PL} \circ \alpha_A^{-1} \circ \alpha_A \equiv \alpha_B \circ f_{PL} \quad (2)$$

Instantiating the commutative diagram for `ExpTree` and `UNTYPEDLAMBDA` yields the diagram in Figure 4. A dashed line indicates a function that is implemented in terms of the other functions in the diagram and/or standard primitives.

We call these isomorphic notional machines because they are isomorphic to the aspect of the programming language they focus on (a condition sometimes called *strong simulation* [27]). Of course that's a rather strong condition and not every notional machine is isomorphic so throughout the next sections we will move further away from this simple example, arriving at other instantiations of this commutative diagram.

2.2 Monomorphic Notional Machines

Notional machines can also serve as the basis for so-called “visual program simulation” [31] activities, where students manually construct representations of the program execution. This effort often is supported by tools, such as interactive diagram editors, that scaffold the student's activity. Obviously, instructors will want to see their students creating correct representations. However, to prevent students from blindly following a path to a solution prescribed by the tool, the visual program simulation environment should also allow *incorrect* representations.

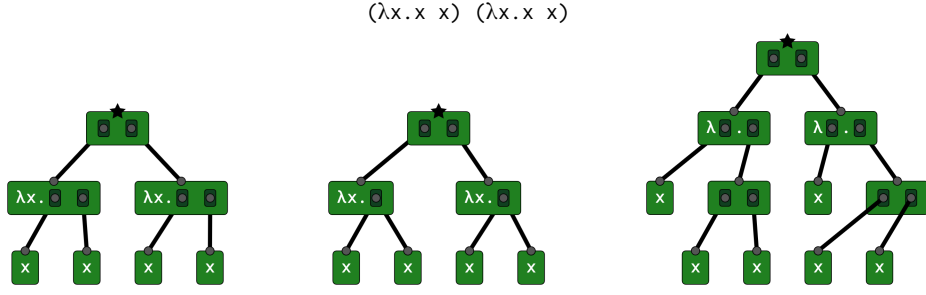


Fig. 5. The omega combinator in UNTYPEDLAMBDA (top) and (incorrect) representations in EXPEDITORDIAGRAM notional machine (bottom).

ExpressionTutor⁵ is such an educational tool to teach the structure, typing, and evaluation of expressions in programming courses. In ExpressionTutor, students can interactively construct expression tree diagrams given a source code expression. The tool is language agnostic so each node can be freely constructed (by the instructor or the student) to represent nodes of the abstract syntax tree of any language. Nodes can contain any number of holes that can be used to connect nodes to each other. Each hole corresponds to a place in an abstract syntax tree node where an expression would go. Nodes can be connected in a variety of ways, deliberately admitting incorrect structures that not only may not be valid abstract syntax trees of a given programming language but may not even be trees. Even the root node (labeled with a star) has to be explicitly labeled by the student, so it is not guaranteed to exist in every diagram.

We define the notional machine EXPEDITORDIAGRAM, which models the behavior of ExpressionTutor. The fact that ExpressionTutor allows students to construct incorrect expression tree diagrams means that the abstraction function α is not bijective, as was the case of EXPEDITOR's α . Such incorrect diagrams do not correspond to programs, thus α is deliberately not surjective.

2.2.1 Illustrative Example. Figure 5 uses EXPEDITORDIAGRAM to represent the omega combinator. The top shows the textual form on the level of the programming language. Below that are three different incorrect representations students could produce. The left tree collapses the applications (the terms $x \ x$) into the enclosing lambda abstraction. The middle tree similarly does this, but it preserves the structure of the lambda abstraction node, while violating the well-formedness of the tree by plugging two children into the same hole. The right tree shows a different problem, where the definition of the name is pulled out of the lambda abstraction and shown as a name use.

2.2.2 Commutative Diagram. In general, if the mapping α_A , from the abstract representation of the programming language (A_{PL}) to the abstract representation of the notional machine (A_{NM}), is an injective but non-surjective function, we can still define the operations on A_{NM} in terms of the operations on A_{PL} . For this we define a function $\alpha_A^\circ :: A_{NM} \rightarrow \text{Maybe } A_{PL}$ to be a left inverse of α_A such that $\alpha_A^\circ \circ \alpha_A \equiv \text{return}$ (we use *return* and *fmap* to refer to the *unit* and *map* operations on monads). Here we modeled the left inverse using a *Maybe* but another monad could be used, for example, to capture information about the values of type A_{NM} that do not have a corresponding value in A_{PL} . The top-right vertex of the square (B_{NM}) in this case is the type *Maybe* B'_{NM} and the mapping α_B can be implemented in terms of a mapping $\alpha'_B :: B_{PL} \rightarrow B'_{NM}$ like so:

⁵expressiontutor.org

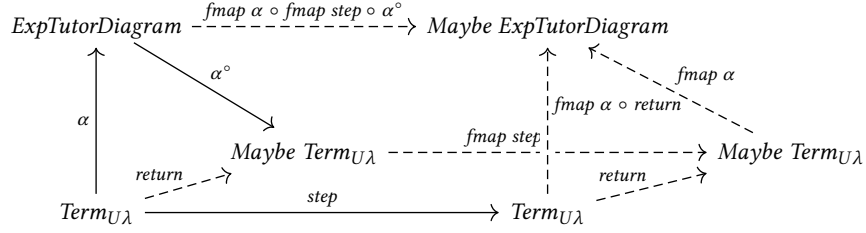


Fig. 6. Instantiation of the commutative diagram in Figure 3 for the notional machine `ExpTutorDiagram`

$$\alpha_B :: B_{PL} \rightarrow B_{NM}$$

$$\alpha_B = \text{return} \circ \alpha'_B$$

Using the left inverse α_A° and α'_B , we define the operation on A_{NM} as follows:

$$f_{NM} :: A_{NM} \rightarrow B_{NM}$$

$$f_{NM} = \text{fmap } \alpha'_B \circ \text{fmap } f_{PL} \circ \alpha_A^\circ$$

This square commutes like so:

$ \begin{aligned} & f_{NM} \circ \alpha_A \\ \equiv & \{ \text{definition of } f_{NM} \} \\ & \text{fmap } \alpha'_B \circ \text{fmap } f_{PL} \circ \alpha_A^\circ \circ \alpha_A \\ \equiv & \{ \alpha_A^\circ \text{ is left inverse of } \alpha_A \} \\ & \text{fmap } \alpha'_B \circ \text{fmap } f_{PL} \circ \text{return} \\ \equiv & \{ \text{third monad law} \} \\ & \text{fmap } \alpha'_B \circ \text{return} \circ f_{PL} \end{aligned} $	$ \begin{aligned} & \alpha_B \circ f_{PL} \\ \equiv & \{ \text{definition of } \alpha_B \} \\ & \text{return} \circ \alpha'_B \circ f_{PL} \\ \equiv & \{ \text{third monad law} \} \\ & \text{fmap } \alpha'_B \circ \text{return} \circ f_{PL} \end{aligned} $
--	--

We can use this result to instantiate the commutative diagram of Figure 3 for `ExpTutorDiagram` and `UNTYPED-LAMBDA`, shown in Figure 6. A_{PL} is defined to be the type `ExpTutorDiagram`, which essentially implements a graph. Each node has a top plug and any number of holes, which contain plugs. Edges connect plugs. That allows for a lot of flexibility in the way nodes can be connected.

The ExpressionTutor tool is language agnostic but we can only talk about soundness of a notional machine with respect to some language and some aspect of that language. In this case, we say the tool implements a family of notional machines, each one for a given aspect of focus and a given programming language. We consider here a notional machine focused on evaluation and the programming language again `UNTYPEDLAMBDA` (denoted again by the type `TermUλ`), with f_{PL} equal to `step`.

We can always construct a mapping α , from `TermUλ` to `ExpTutorDiagram`, because every term $t :: \text{Term}_{U\lambda}$ forms a tree and every ExpressionTutor diagram $d :: \text{ExpTutorDiagram}$ forms a graph. For each possible term in `TermUλ`, we need to define a pattern for the content of the corresponding `ExpTutorDiagram` node which will help the student identify the kind of node. The construction of the left inverse mapping $\alpha^\circ :: \text{ExpTutorDiagram} \rightarrow \text{Maybe Term}_{U\lambda}$ requires more care. We need to make sure that the diagram forms a proper tree and that the pattern formed by the contents of each

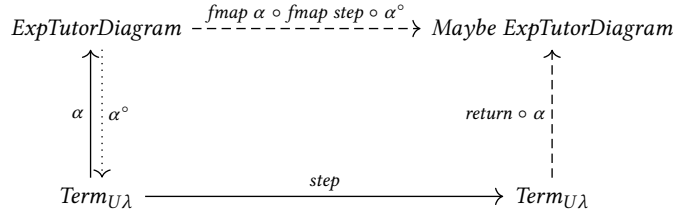


Fig. 7. Simplified version of the commutative diagram for `ExpTutorDiagram` shown in Figure 6.

`ExpTutorDiagram` node corresponds to a possible $Term_{U\lambda}$ node, besides making sure that they are connected in a way that indeed corresponds to a valid $Term_{U\lambda}$ tree.

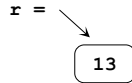
In the next section, we construct another commutative diagram where f_{NM} is defined using f_{PL} and a left inverse mapping α_A° . To emphasize that point and simplify the diagrams, we will depict the left inverse in the diagram as a dotted line pointing from A_{NM} to A_{PL} (even though α_A° is of type $A_{NM} \rightarrow Maybe\ A_{PL}$ and not $A_{NM} \rightarrow A_{PL}$) and omit the path via $Maybe\ A_{PL}$ as shown in Figure 7.

We call these monomorphic notional machines because there is a monomorphism (injective homomorphism) between the notional machine and the aspect of the programming language it focuses on. This is the case here by design, to allow students to make mistakes by constructing wrong diagrams that don't correspond to programs. In general, this will be the case whenever there are values of A_{NM} (the abstract syntax of the notional machine) that have no correspondence in the abstract representation of the language (A_{PL}). That's often the case in memory diagrams [13, 15, 23] (notional machines used to show the relationship between programs and memory) because they typically allow for the representation of memory states that cannot be produced by legal programs. We show an example of such a notional machine in the next section.

2.3 A Monomorphic Notional Machine to Reason About State

A common use of notional machines is in the context of reasoning about state. In fact, 16 of the 37 notional machines in the dataset by Fincher et al. [19] focus on either References, Variables, or Arrays (see Section 5). An example of the use of a visual notation to represent state can also be found in TAPL [28, p. 155]. In Chapter 13 ("References"), the book extends the simply typed lambda-calculus with references (a language we will refer to as `TYPEDLAMBDAREF`⁶). It explains references and aliasing by introducing a visual notation to highlight the difference between a *reference* and the *cell* in the store that is pointed to by that reference. We will refer to this notation, which we will develop into a notional machine, as `TAPLMemoryDiagram`. In this notation, references are represented as arrows and cells are represented as rounded rectangles containing the representation of the value contained in the cell. Before designing the notional machine, we need to see the context in which this notation is used in the book.

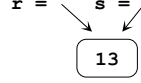
The book first uses this notation to explain the effect of creating a reference. It shows that when we reduce the term `ref 13` we obtain a reference (a store location) to a store cell containing 13. The book then represents the result of binding the name `r` to such a reference with the following diagram:



⁶The syntax and reduction rules for `TYPEDLAMBDAREF` are reproduced in the appendix provided as supplementary material.

In the book, this operation is written as $r = \text{ref } 13$, but as we will see in the next Section, this form of name binding (name = term) exists only in a REPL-like context which is not part of the language.

The book continues explaining that we can “make a copy of r ” by binding its value to another variable s (with $s = r$) and shows the resulting diagram:



The book then explains that one can verify that both names refer to the same cell by *assigning* a new value to s and reading this value using r (for example, the term $s := 82; !r$ would evaluate to 82). Right after, the book suggests to the reader an exercise to “draw a similar diagram showing the effects of evaluating the expressions $a = \{\text{ref } 0, \text{ref } 0\}$ and $b = (\lambda x:\text{Ref Nat}.\{x, x\}) (\text{ref } 0)$.” Although we understand informally the use of this diagram in this context, how can we know what a correct diagram would be in general for any given program? This is what we aim to achieve by designing a notional machine based on this notation.

Let’s see how we would turn that kind of diagram into a sound notional machine. We want to construct a commutative diagram where A_{PL} is an abstract representation of the state of a `TYPEDLAMBDA REF` program execution, A_{NM} is an abstract representation of the diagram presented in the book, and f_{PL} is an operation that affects the state of the store during program execution.

In a first attempt, let’s choose f_{PL} to be an evaluation step and A_{PL} to be modeled as close as possible to the presentation of a `TYPEDLAMBDA REF` program as described in the book. In that case, A_{PL} is the program’s abstract syntax tree together with a *store*, a mapping from a location (a reference) to a value.

2.3.1 Problem: Beyond the Language. The first challenge is that the name-binding mechanism used in the examples above (written as name = term) exists only in a REPL-like context in the book used for the convenience of referring to terms by name. It is actually not part of the language (`TYPEDLAMBDA REF`) so it is not present in this representation of A_{PL} and as a result it cannot be mapped to A_{NM} (the notional machine). We will avoid this problem by avoiding this name-binding notation entirely and writing corresponding examples fully in the language. The only mechanism actually in the language to bind names is by applying a lambda to a term. Let’s see how we can write a term to express the behavior described in the example the book uses to introduce the diagram (shown earlier), where we:

- (1) Bind r to the result of evaluating $\text{ref } 13$
- (2) Bind s to the result of evaluating r
- (3) Assign the new value 82 to s
- (4) Read this new value using r

Using only the constructs in the language, we express this with the following term:

$$(\lambda r:\text{Ref Nat}.\ (\lambda s:\text{Ref Nat}.\ s := 82; !r) \ r) \ (\text{ref } 13)$$

2.3.2 Problem: Direct Substitution. The problem now is that if we model A_{PL} and evaluation as described in the book, the result of reducing a term $(\lambda x.t_1) t_2$ is the term obtained by replacing all free occurrences of x in t_1 by t_2 (modulo alpha-conversion), so we don’t actually keep track of name binding information. What we have in A_{PL} at each step is an abstract syntax tree and a store, but we have no information about which names are bound to which values because the names were already substituted in the abstract syntax tree. That would be enough to do Exercise 13.1.1, for example, whose solution is shown in Figure 8. But the absence of explicit information mapping names to values makes it less

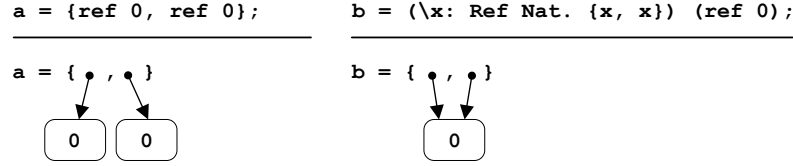


Fig. 8. TAPLMemoryDiagram for TypedLambdaRef for TAPL Exercise 13.1.1

suitable to talk about aliasing, because even though a term may contain, at any given point during evaluation, multiple occurrences of the same location, it is not possible to know if these locations correspond to different names, and one may need to trace several reduction steps back to find out when a name was substituted by a location.

We need to change Ap_L and $step$ to capture this information, keeping not only a store but an explicit name environment that maps names to values, and only substituting the corresponding value when we evaluate a variable. Like the definition of application in terms of substitution, we have to be careful to avoid variable capture by generating fresh names when needed.

2.3.3 Illustrative Example. Figure 9 shows two variations of the notional machine being used to explain the evaluation of the term we had described before. It shows the state after each reduction step, on the left without an explicit name environment and on the right with an explicit name environment. Between each step, a line with the name of the applied reduction rule is shown. Notice that the representation with a name environment requires extra name lookup steps.

In both variations, the representation of the program (the term) being evaluated appears first (with gray background). Each term is actually an abstract syntax tree, which we represent here, like in the book, with a linearized textual form. Location terms are represented as arrows starting from where they appear in the abstract syntax tree and ending in the store cell they refer to.

The naming environment is shown as a table from variable names to terms. Store cells also contain terms. This means the textual representation of terms that appear both inside name environments and inside cells may also contain arrows to other cells.

2.3.4 Commutative Diagram. Similar to the abstract representation of the program execution, the abstract representation of the notional machine contains three parts: one for the program being evaluated, one for the name environment, and one for the store.

```

data TAPLMemoryDiagram l = TAPLMemoryDiagram {
  memDiaTerm :: DTerm l,
  memDiaNameEnv :: Map Name (DTerm l),
  memDiaStore :: Map (DLocation l) (DTerm l)
data DTerm l = Leaf String
              | Branch [DTerm l]
              | TLoc (DLocation l)

```

The type $DLocation$ corresponds to arrow destinations (arrow endpoints). A term is represented as a rose tree of *Strings* augmented with a case for location.

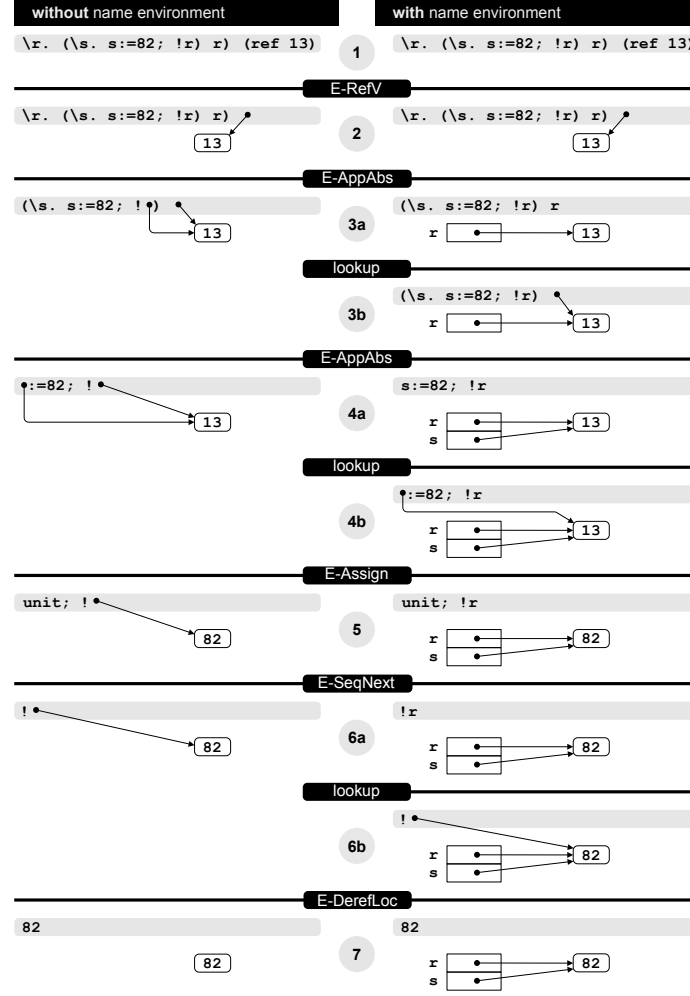


Fig. 9. Tracing the evaluation of $(\lambda r:\text{Ref Nat}. (\lambda s:\text{Ref Nat}. s := 82; !r) r) (\text{ref } 13)$ using TAPLMemoryDiagram.

The concrete representation of a $DTerm$ can be in linearized text form or as a tree akin to that shown in Section 2.1. The representation of the nodes in a $DTerm$ tree that are $TLoc$ are shown as arrow starting points. These arrows end in the cell corresponding to the $DLocation$ in each $TLoc$. The concrete representation of the store relates the visual position of each cell with the $DLocation$ of each cell. That leads to the commutative diagram in Figure 10, where we use the symbol $\circ_M$ to denote monadic function composition (the fish operator $\leq=<$ in Haskell).

2.4 A Monomorphic Notional Machine to Reason About Types

So far we have seen examples of commutative diagrams where f_{PL} is *step* (a function that performs a reduction step) but in principle, f_{PL} could be any operation on A_{PL} that is the focus of a given notional machine. Let's look at an example of notional machine where we do not focus on evaluating but on typing an expression. The language this notional

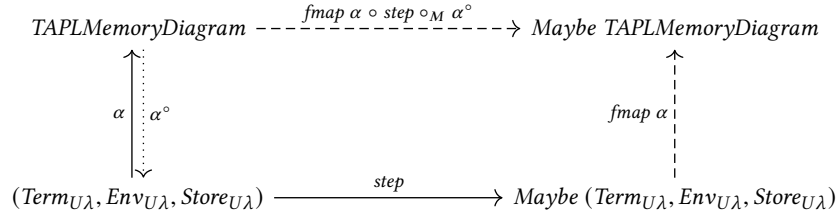


Fig. 10. Instantiation of the commutative diagram in Figure 3 for the notional machine TAPLMemoryDiagram and the programming language TYPEDLAMBDARef.

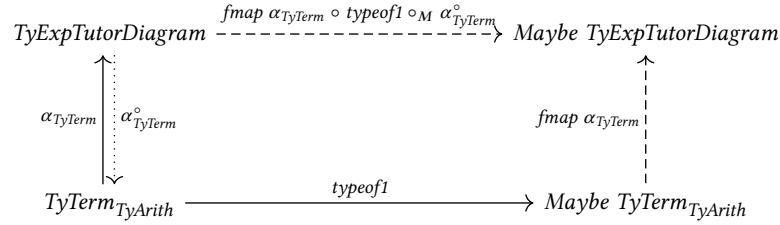


Fig. 11. Instantiation of the commutative diagram in Figure 3 for a notional machine focused on type-checking programs in TYPEDARITH. The notional machine exposes the inner workings of the typing algorithm.

machine focuses on is TYPEDARITH⁷, a language of typed arithmetic expression, which is the simplest typed language introduced in TAPL [28, p. 91]. We describe two designs.

In the first design, the data type used for the abstract representation of the notional machine (A_{NM}) is *ExpTutorDiagram*, used in Section 2.2. We represent a program in TYPEDARITH with the type $\text{Term}_{\text{TyArith}}$ and the operation we focus on (f_{PL}) is $\text{typeof} :: \text{Term}_{\text{TyArith}} \rightarrow \text{Maybe Type}_{\text{TyArith}}$, a function that gives the type of a term (for simplicity we use a *Maybe* here to capture the cases where a term is not well-typed).

We then have two abstraction functions:

$$\alpha_{\text{Term}} :: \text{Term}_{\text{TyArith}} \rightarrow \text{ExpTutorDiagram}$$

$$\alpha_{\text{Type}} :: \text{Type}_{\text{TyArith}} \rightarrow \text{Type}_{\text{ExpTutor}}$$

The implementation of f_{TyArith} , the notional machine operation (f_{NM}) that produces a notional machine-level representation of maybe the type of a term, is analogous to what is shown in Figure 10 because (1) the abstraction function (α_{Term}) has a left inverse ($\alpha_{\text{Term}}^\circ$) and (2) f_{PL} produces a *Maybe*.

$$f_{\text{TyArith}} :: \text{ExpTutorDiagram} \rightarrow \text{Maybe Type}_{\text{ExpTutor}}$$

$$f_{\text{TyArith}} = \text{fmap } \alpha_{\text{Type}} \circ \text{typeof} \circ_M \alpha_{\text{Term}}^\circ$$

As is, a student may benefit from the notional machine's representation of the program's abstract syntax tree and that may be helpful to reason about typing but the notional machine does not expose to the student the inner workings of the process of typing a term.

⁷The syntax and typing rules for TYPEDARITH are reproduced in the appendix provided as supplementary material.

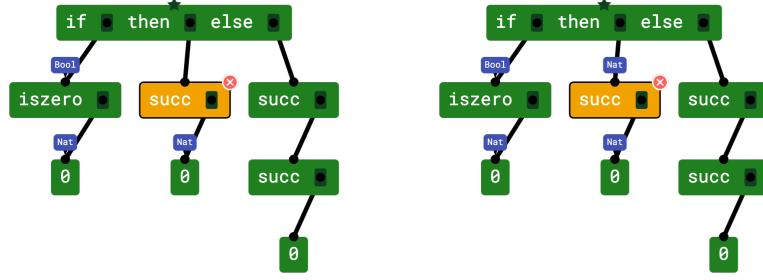


Fig. 12. Representation of one step in the notional machine `TYPEDEXP_TUTOR_DIAGRAM` as it types the term `if iszero 0 then succ 0 else succ succ 0` in the language `TYPEDARITH`.

The second design, represented in the diagram in Figure 11, tackles this issue by enriching the notional machine in a way that allows it to go step-by-step through the typing algorithm. The idea is that f_{NM} now does not produce a type but gradually labels each subtree with its type as part of the process of typing a term. For this, A_{NM} here is *TyExpTreeDiagram*, which differs from *ExpTreeDiagram* by adding to each node a possible type label. We still want to write f_{NM} in terms of f_{PL} . The key insight that enables this is to change f_{PL} from *typeof* to *typeof1*. The difference between *typeof* and *typeof1* is akin to the difference between big-step and small-step semantics: *typeof1* applies a single typing rule at a time. As a result, we have to augment our representation of a program by bundling each term with a possible type (captured in type $TyTerm_{TyArith}$). The abstraction function and its left inverse are updated accordingly. The resulting notional machine is depicted in Figure 12.

Interestingly, given an expression e , once we label all nodes in the *ExpressionTutor* diagram of e with their types, the depiction of the resulting diagram is similar to the typing derivation tree of e .

Note that types are themselves trees but here we're representing them in a simplified form as textual labels because the primary goal of *ExpressionTutor* is to represent the structure of terms, not the structure of types.

3 Analyzing Existing Notional Machines

So far we have seen how we can design sound notional machines by constructing f_{NM} using f_{PL} and functions that convert between A_{PL} and A_{NM} . Now we will analyze existing notional machines, using their informal description to construct all components of the commutative diagram. In particular, here, we have a description of f_{NM} entirely in terms of A_{NM} . We then use property-based testing, using the soundness condition as a property, to uncover unsoundnesses and suggest improvements that eliminate them. Table 2 shows the notional machines we use in this section as well as the corresponding programming language and aspect of the semantics of the programming language that the notional machine focuses on.

Section	Notional Machine	Programming Language	Focus
3.1	ALLIGATOR	UNTYPEDLAMBDA	Evaluation
3.2	LISTASSTACK	-	Data Structure (List)
3.3	ARRAYASPARKINGSPOTS	JAVA	Evaluation (Arrays)

Table 2. Notional machines, programming languages, and aspects of focus used in Section 3.

3.1 Debugging a Notional Machine: The Case of Alligator Eggs

Alligator Eggs⁸ is a game conceived by Bret Victor to introduce the lambda-calculus in a playful way. It is essentially a notional machine for the untyped lambda-calculus. The game has three kinds of pieces and is guided by three rules.

Pieces. The pieces are *hungry alligators*, *old alligators*, and *eggs*. Old alligators are white, while hungry alligators and eggs are colored with colors other than white. The pieces are placed in a plane and their relative position with respect to each other determines their relationship. All pieces placed under an alligator are said to be guarded by that alligator. An alligator together with the pieces that may be guarded by it form a family. Families placed to the right of another family may be eaten by the guardian of the family on the left, depending on the applicability of the gameplay rules. Every egg must be guarded by an alligator with the same color (this must be a hungry alligator because eggs cannot be white).

Rules. There are three rules that determine the “evolution of families” over time:

Eating rule If there is a family guarded by a hungry alligator in the plane and there is a family or egg to its right, then the hungry alligator eats the entire family (or egg) to its right and the pieces of the eaten family are removed. The alligator that ate the pieces dies and the eggs that were guarded by this alligator and that have the same color of this alligator are hatched and are replaced by a copy of what was eaten by the alligator.

Color rule Before a hungry alligator A can eat a family B , if a color appears both in A ’s proteges and in B , then that color is changed in one of the families to another color different from the colors already present in these families.

Old age rule If an old alligator is guarding only one egg or one family (which itself may be composed of multiple families), then the old alligator dies and is removed.

Relation to the Untyped Lambda-Calculus. According to their description, the way ALLIGATOR relates to the untyped lambda-calculus is as follows: “A hungry alligator is a lambda abstraction, an old alligator is parentheses, and eggs are variables. The eating rule corresponds to beta-reduction. The color rule corresponds to (over-cautious) alpha-conversion. The old age rule says that if a pair of parentheses contains a single term, the parentheses can be removed”. Although very close, this relation is not completely accurate. We will identify the limitations and propose solutions.

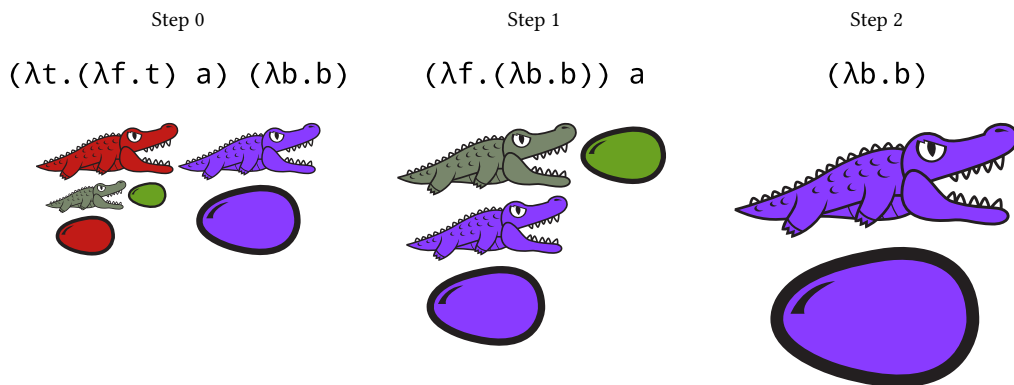


Fig. 13. Evaluation of $(\lambda t.(\lambda f.t) a) (\lambda b.b)$ in the untyped lambda calculus (top) and ALLIGATOR notional machine (bottom).

⁸<http://worrydream.com/AlligatorEggs/>

3.1.1 Illustrative Example. Figure 13 shows a representation of evaluating the term $(\lambda t. (\lambda f. t) \ a) \ (\lambda b. b)$ using the ALLIGATOR notional machine. Step 0 shows the alligator families corresponding to the term. In the first step, the red alligator eats the family made of the purple alligator and the purple egg. As a result, the eaten family disappears, the red egg guarded by the red alligator hatches and is replaced by the family that was eaten, and the red alligator disappears (dies). In the second step, the grey alligator eats the green egg. As a result, the grey alligator dies and no eggs are hatched because it was not guarding any grey eggs. We are left with the purple family.

3.1.2 Commutative Diagram. To build a commutative diagram for ALLIGATOR, we need to build the abstract representation of the notional machine A_{NM} , which corresponds to the game pieces and the game board, the abstraction function $\alpha :: Term_{U\lambda} \rightarrow A_{NM}$, and an f_{NM} function, which correspond to the rules that guide the evolution of alligator families. First, we look more precisely at the game pieces and their relationship with $Term_{U\lambda}$ to model A_{NM} .

Eggs An egg corresponds to a variable use and its color corresponds to the variable name.

Hungry alligators A hungry alligator somewhat corresponds to a lambda abstraction with its color corresponding to the name of the variable introduced by the lambda (a variable definition) and the pieces guarded by the hungry alligator corresponding to the body of the lambda abstraction. But differently from a lambda abstraction, a hungry alligator does not have to be guarding any pieces, which has no direct correspondence with the lambda calculus because a lambda abstraction cannot have an empty body.

Old alligators At first glance, an old alligator seems to correspond to parenthesis. But that is not exactly the case. The lambda abstraction in the term $(\lambda t. \lambda f. t) \ a \ b$ requires parentheses because conventionally the body of a lambda abstraction extends as far to the right as possible. However, the corresponding alligator families shown in Figure 13 don't require an old alligator. On the other hand, if we want to represent the term $a \ (b \ c)$, then we need an old alligator. Figure 14 shows an example of a term that requires an old alligator.

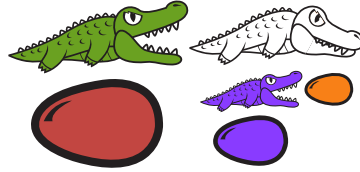


Fig. 14. Term $(\lambda a. y) \ ((\lambda b. b) \ c)$ requires an old alligator.

Now let's look at the terms of the untyped lambda-calculus. If hungry alligators are lambda abstractions and eggs are variables then what is an application? Applications are formed by the placement of pieces on the game board. When an alligator family or egg (corresponding to a term t_1) is placed to the left of another family or egg (corresponding to a term t_2), then this corresponds to the term t_1 applied to t_2 (in lambda calculus represented as $t_1 \ t_2$).

Notice that because every egg must be guarded by a hungry alligator with the same color, strictly speaking, an egg cannot appear all by itself. That corresponds to the fact that in the untyped lambda-calculus only lambda terms are values, so a term cannot have unbound variables. Textbooks, of course, often use examples with unbound variables but these are actually metavariables that stand for an arbitrary term. So, for convenience, we will consider that an egg by itself also forms a family.

$$\begin{array}{ccc}
[AlligatorFamily] & \xrightarrow{f_{NM}} & [AlligatorFamily] \\
\uparrow \alpha & & \uparrow \alpha \\
Term_{U\lambda} & \xrightarrow{step} & Term_{U\lambda}
\end{array}$$

Fig. 15. First attempt at instantiating the commutative diagram in Figure 3 for the notional machine ALLIGATOR.

We can then model an alligator family with the type *AlligatorFamily*, and a game board as simply a list of alligator families. The result is the commutative diagram shown in Figure 15.

```

data AlligatorFamily =
  HungryAlligator Color [AlligatorFamily]
  | OldAlligator [AlligatorFamily]
  | Egg Color

```

The abstraction function α relies on some function (*n2c*) to map from names to colors.

```

α :: TermUλ → [AlligatorFamily]
α (Var name) = [Egg (n2c name)]
α (Lambda name e) = [HungryAlligator (n2c name) (α e)]
α (App e1 e2@(App _ _)) = α e1 # [OldAlligator (α e2)]
α (App e1 e2) = α e1 # α e2

```

From Proof to Property-Based Testing. The commutativity of the diagrams presented in Chapter 2 was demonstrated using equational reasoning. Here instead, we implement the elements that constitute the commutative diagram and use property-based testing to test if the diagram commutes. This approach is less formal and it does not prove the notional machine correct, but it is lightweight and potentially more attractive to users that are not familiar with equational reasoning or mechanised proofs. We will see here that, despite its limitations, this approach can go a long way in revealing issues with a notional machine. The idea is that a generator generates terms $t_i :: Term_{U\lambda}$ and checks that $(f_{NM} \circ \alpha) t_i \equiv (\alpha \circ step) t_i$.

de Bruijn Alligators. The first challenge is that we need to compare values of type $[AlligatorFamily]$ that were produced using f_{NM} with values produced using $step$. As we have seen, the colors in *AlligatorFamily* correspond to variable names but the way $step$ generates fresh names (which then are turned into colors) may be different from the way f_{NM} will generate fresh colors. In fact, the original description of ALLIGATOR anticipates the challenge of comparing alligator families. In the description of possible gameplays, they clarify that to compare alligator families we need to take into account that families with the same "color pattern" are equivalent. This can be achieved by using a *de Bruijn representation* [5] of Alligators. We turn *AlligatorFamily* into *AlligatorFamilyF Color* and before comparing families we transform them into *AlligatorFamilyF Int* following the de Bruijn indexing scheme. The commutative diagram we are moving towards is shown in Figure 16.

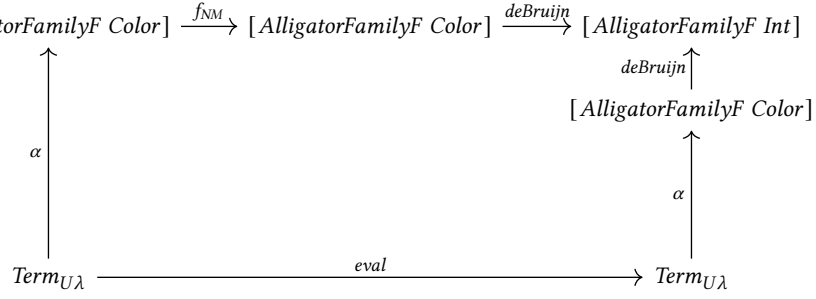


Fig. 16. Second attempt at instantiating the commutative diagram in Figure 3 for the notional machine ALLIGATOR.

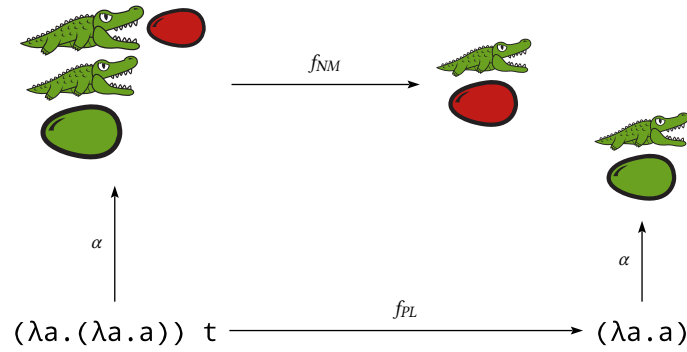


Fig. 17. Unsoundness: *bound* occurrences of a variable should not be substituted.

Evaluation Strategy. With this setup in place, the next step is to implement f_{NM} in terms of the game rules. The eating rule (together with the color rule) somewhat corresponds to beta-reduction but under what evaluation strategy? The choice of evaluation strategy turns out to affect not only the eating rule but also the old age rule. According to the original description, any hungry alligator that has something to eat can eat and one of the original examples shows a hungry alligator eating an egg even when they are under another hungry alligator. That would correspond to a *full beta-reduction* evaluation strategy but we will stick to a call-by-value lambda-calculus interpreter so we will adapt the rules accordingly. The old age rule has to be augmented to trigger the evolution of an old alligator family that follows a topmost leftmost hungry alligator and families under a topmost leftmost old alligator. The eating rule should be triggered only for the topmost leftmost hungry alligator, unless it is followed by an old alligator (in which case the augmented old age rule applies). The color rule plays an important role in the correct behavior of the eating rule as a correspondence to beta-reduction. That's because indeed "the color rule corresponds to (over-cautious) alpha-conversion", so it is responsible for avoiding variable capture.

With all the rules implemented, we can define a function *evolve* that applies them in sequence. We will then use *evolve* in the definition of f_{NM} . One application of *evolve* corresponds to one step in the notional machine layer but that step does not correspond to a step in the programming language layer. For example, The main action of the old age rule (to remove old alligators) does not have a correspondence in the reduction of terms in UNTYPEDLAMBDA. In terms of simulation theory, in this case the simulation of the programming language by the notional machine is not lock-step.

To adapt our property-based testing approach, instead of making f_{PL} equal to $step$, we will simply reduce the term all the way to a value (leading to the use of $eval$ as f_{PL} in Figure 16) and correspondingly define f_{NM} to be the successive applications of $evolve$ until we reach a fixpoint.

3.1.3 Problem: Substitution of Bound Variables. Now we have all the building blocks of the commutative diagram. We can put them together by running the property-based tests to try to uncover issues in the diagram and indeed we do. According to the eating rule, after eating, a hungry alligator dies and if it was guarding any eggs of the same color, each of those eggs hatches into what was eaten. So the family corresponding to $(\lambda a. (\lambda a. a)) \ t$, for example, would evolve to the family corresponding to $\lambda a. t$ instead of $\lambda a. a$, as shown in Figure 17. This issue corresponds to a well-known pitfall in substitution: we cannot substitute *bound* occurrences of a variable, only the ones that are *free*.

The issue can be solved in one of the following ways:

- (1) Refining the description of the eating rule, changing “if she was guarding any eggs of the same color, each of those eggs hatches into what she ate” into “if she was guarding any eggs of the same color **that are not guarded by a closer alligator with the same color**, each of those eggs hatches into what she ate”;
- (2) Restricting all colors of hungry alligators in a family to be distinct;
- (3) Defining colors to correspond to de Bruijn indices instead of names. This means that not only colors wouldn’t be repeated in the same family but also that every family would use the same “color scheme” for structurally equivalent terms.

3.2 Notional Machines for Data Structures

Although most notional machines focus on the semantics of programming language constructs, some notional machines instead focus on data structures. One such notional machine, which we model in this section, is the “List as Stack of Boxes” (Figure 18), described by Du Boulay and O’Shea [17] and included in the dataset of notional machines analyzed by Fincher et al. [19].

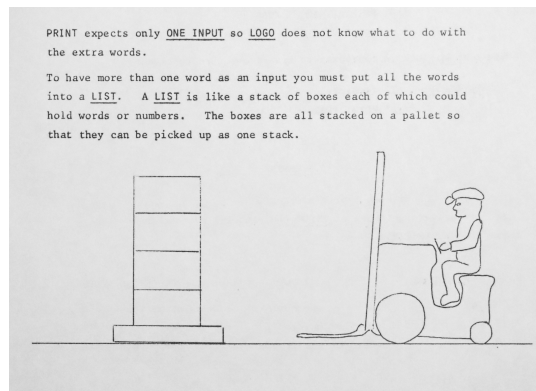


Fig. 18. The “List as Stack of Boxes” notional machine as described by Du Boulay and O’Shea [17].

For notional machines focussing on the dynamic semantics of a language ($ExpTree$, $ExpTree$, $TAPLMemoryDiagram$), the type A_{PL} represents a program in the language under focus (and in the case of $TAPLMemoryDiagram$, additional information needed to evaluate the program) and the function f_{PL} performs an evaluation step. In the case of

TYPEDExpTUTORDiagram, which focusses on type-checking (the static semantics of TYPEDArith), A_{PL} also represents a program in that language and f_{PL} performed type-checking. Now we will model a notional machine focussing on a data structure, so A_{PL} represents that data structure and there are several f_{PL} functions, one for each operation supported by that data structure. The type A_{NM} can be seen as an abstraction of A_{PL} and it should provide corresponding operations. The commutation of the diagram demonstrates the correctness of this abstraction.

Modelling “List as Stack of Boxes” requires some adaptations to the original description:

- (1) To avoid partiality of the operations typically used to access the head and tail of a list (FIRST and REST in the original description), we define a list using three operations: *Empty* and *Cons* to construct a list and *uncons* :: $List\ a \rightarrow Maybe\ (a, List\ a)$ to deconstruct it. That change also has a convenient representation in the notional machine.
- (2) In the stack of boxes, each value is shown as a *String* in a box, which means we can only represent lists of values for which we can create a *String* representation.
- (3) The original description wasn’t explicit about the representation of an empty list. We need a corresponding empty stack of boxes that can be treated as a stack and not just the absence of boxes. For that we will use a pallet (used to hold boxes in storage). The original description mentions a pallet in two contexts:
 - (a) “boxes are stacked on a pallet so that they can be picked up as one stack”. That’s an important part of the notional machine’s behavior but we also need to be able to pick up a box from the top of the stack;
 - (b) “The beginning of a list, the top of the stack, is marked with [and the end of the list, the pallet is marked with]”. Here there’s an asymmetry between the end of the list, represented with a pallet, and the beginning of the list, which has no representation in the notional machine. Our denotation of pallet is different: it is the representation of an empty list.

The result is that we can construct lists either with a pallet or by stacking a box on top of a stack, which must have a pallet at the end. To deconstruct it, we pick up a box from the top of the stack (*pickUp* :: $Stack \rightarrow Maybe\ (Box, Stack)$). When trying to pick up a box, either there is only the pallet, in which case we get nothing, or there is a box on top of a stack, in which case we are left with the box and the rest of the stack.

The process of formalizing the notional machine, making it precise, once again helped to identify the issues and improve the notional machine.

3.3 A Notional Machines for Arrays

With the techniques we have developed so far, we can turn again to the notional machine “Array as Row of Parking Spaces is Parking Lot”, presented in Section 1.1. There we considered Java as the underlying programming language and already identified the first issue, caused by the difference between the representation of arrays of primitive types and arrays of reference types. Here, we will make the notional machine more precise and in the process resolve two problems.

One approach would be to model the subset of Java needed for the notional machine to work, but Java is a fairly complex language. Instead we start by following the approach taken in the last section: we model the PL layer as an idealized array and the operations it supports. An array supports three operations: *allocating* an array of a given type and a given length, *reading* an element at a given index, and *writing* an element to a given index.

3.3.1 Problem: reference types versus primitive types. One of the appeals of the notional machine is that we would represent a newly allocated array of objects as an empty parking lot, because it does not contain “valid” values (their

slots contain *null*). The first issue is that arrays of primitive types are really different in that, when newly allocated, their slots already contain valid values. These arrays would be represented as fully booked parking lots with slots that can never be empty! Here, instead of changing the design of the notional machine, we can use this misfit in the metaphor as a learning opportunity and explicitly discuss it with students. There is still a decision to be made about how exactly the values in the array are going to be represented. We could represent a value as a car with the string representation of the value drawn on its roof, but representing both unboxed and boxed versions of an integer with the same string could be confusing. Instead, we could opt for representing values of reference types as cars of a different color, for example. Of course an array can also contain other arrays, which are themselves objects, and that complicates the picture.

3.3.2 Problem: representing multiple arrays. To represent multiple arrays, and the relationship between them, we cannot model the PL layer simply as an idealized array⁹. Instead, we use the programming language LAMBDA_{REF} augmented with arrays. We first used this language in Section 2.3, where the TAPLMemoryDiagram notional machine represented all constructs of the language as well as the program's state as it ran (its memory): the store and name environment. Here we want to represent only arrays and values so the notional machine layer contains, besides the parking lot, essentially a sequence of statements equivalent to the occurrences of all terms in the program that manipulate arrays (an array allocation, array access, and assignment of a value to an array slot) as they happen when the program runs. This way we can ignore all other terms in the language and aspects of the program's memory as it runs. Besides the information present in the array-manipulating terms themselves, we need one more thing: to be able to uniquely identify each array (e.g. with its location, or address). We annotate each parking lot with the corresponding location (for example using an @ and the location identifier) which we use to identify the parking lot when we write to (or read from) its slots.

We can then represent values of reference type using their locations. We could instead use arrows, like we have done in TAPLMemoryDiagram, but that would not work for *ref*'s because here we are only representing arrays.

4 Abstract vs. Concrete Syntax of Notional Machines

When reasoning about a notional machine, it's important to distinguish between the data types that represents the notional machine (the information present in A_{NM} and B_{NM}) and how the notional machine is visualized. This difference is akin to the distinction between the concrete syntax and the abstract syntax of a language. Like a programming language, a notional machine has a concrete and an abstract syntax as well. We also refer to those as concrete and abstract representations of a notional machine. Notice that the concrete representation of a notional machine may not only be a diagram or image that can be depicted on paper but it could also be made of physical objects or enacted with students. In fact, many notional machines are ludic in nature or are built around a metaphor, so the concrete representation of a notional machine is very important.

Figure 19 shows the different layers at play here. On the programming language side, the *parse* function converts from the concrete to the abstract syntax of the language. The function α maps from language constructs to notional machine constructs. On the notional machine side, the function *toDiagram* maps from the abstract representation of the notional machine to its concrete representation, e.g., in the form of an actual diagram. In the case of ALLIGATOR, we use the *diagrams* library [35, 36] to construct the concrete representation so *toDiagram* produces a value of type *Diagram B*, where the type parameter *B* (for Backend) determines the output format of the diagram (e.g. SVG). In fact,

⁹ The representation of lists shown in Section 3.2 is not suitable to represent lists that contain lists.

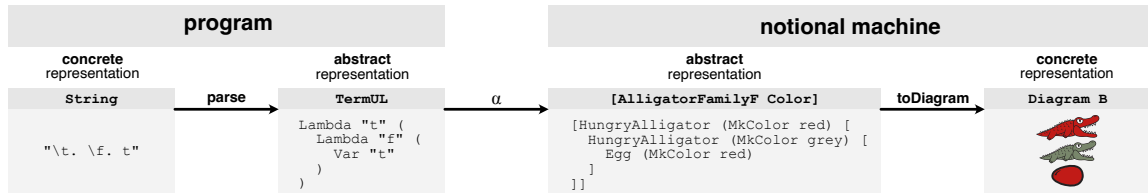


Fig. 19. Both program and notional machine have abstract and concrete representations.

the depictions of Alligator Eggs shown here are generated by calls to our artifact that are embedded directly into the paper.

By decoupling the abstract from the concrete representation, a notional machine can have multiple concrete representations. Alligator Eggs, for example, also describes another concrete syntax that it calls “Schematic Form”. This concrete representation is suitable for working with the notional machine using pencil and paper. Figure 20 shows the Church numeral two (the term $\lambda f. \lambda x. f (f x)$), represented using both concrete representations. In the schematic representation, colors are presented with variable names. An alligator is drawn as a line ending with a $<$ for a mouth, and is preceded by a variable name corresponding to its color. An old alligator is drawn with a line without a mouth. An egg is drawn just with the variable name corresponding to its color.

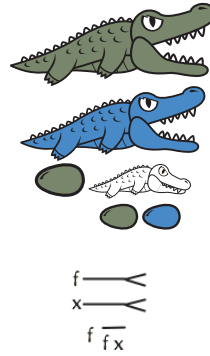


Fig. 20. Multiple concrete representations.

5 Evaluation

The notional machines we presented not only exemplify how to use our framework to reason about notional machines but also were chosen to be representative of the design space of notional machines used in practice. To characterize this design space we analyzed the notional machines in the dataset collected by Fincher et al. [19] and classified them according to three dimensions: form, focus, and language construct. For each dimension, we present the categories of the dimension and for each category we show the number of notional machines in that category, the percentage that number represents of the total, and the sections of the paper containing notional machines that fall into that category. The notional machines are often not precisely described so we often had to make assumptions about their characteristics, how they relate to an underlying programming language, and how they are used to teach programming concepts.

Form	Num	Perc.	Covered
Metaphorical	22	56.41%	3.2 3.3 3.1
Diagrammatic	16	41.03%	2.1 2.2 2.3 2.4
Both	1	2.56%	-
Total	37	100.00%	-

Table 3. Notional machines in the dataset published by Fincher et al. [19] classified according to their form.

The form (Table 3) of a notional machine can be metaphorical (primarily inspired by and represented with real world objects) or diagrammatic (represented with diagrams). The distinction between the two is not always clear-cut, given that a notional machine may mix both forms and some real world objects can be represented diagrammatically. Nevertheless, this distinction is useful because metaphorical notional machines may be more difficult to be made sound, given the existing degrees of freedom and constraints of the real world objects they are inspired by. We used our approach to reason about notional machines of the two forms.

Most notional machines in the dataset focus (Table 4) on the runtime semantics (Evaluation) of a programming language construct (or set of conceptually related constructs) so we further break down these notional machines into the constructs they are primarily focused on (Table 5). Some entries in the table refer to sets of related constructs. For example, the category Control Flow encompasses constructs like loops and conditional statements, which primarily affect control flow. Here the classification is also not clear-cut, not only because a notional machine may focus on multiple constructs but also because some constructs are related to others. For example, notional machines that focus on functions often (although not always) represent variables as well but only the ones that solely (or primarily) focus on variables are classified as such. The category Misc includes five notional machines each of which focuses on a different construct: String literal, Procedure (side-effecting functions), Objects, Instructions (lower level operations), and one that is not clear from the description. The notional machines we reasoned about using our approach cover the majority of these dimensions.

Focus	Num	Perc.	Covered
Evaluation	32	82.05%	2.1 2.2 2.3 3.3 3.1
Type-checking	2	5.13%	2.4
Parsing	2	5.13%	no
Data Structure	2	5.13%	3.2
Logic Gates	1	2.56%	no
Total	37	100.00%	-

Table 4. Notional machines in the dataset published by Fincher et al. [19] classified according to their focus.

Construct	Num	Perc.	Covered
References	8	20.51%	2.3
Functions	5	12.82%	2.1 2.2
Variables	4	10.26%	2.1 2.2 2.3
Arrays	4	10.26%	3.3
Methods	3	7.69%	no
Control Flow	2	5.13%	no
Expressions	1	2.56%	2.1 2.2
Misc	5	12.82%	no
Total	32	82.05%	-

Table 5. Notional machines that focus on evaluation broken down by the set of construct they focus on.

5.1 Expressing Complex Language Semantics

Although the notional machines presented here as well as in the dataset by Fincher et al. [19] focus on aspects of program semantics that one may consider simple, the framework we presented can be used to reason about more

complex aspects of program semantics. An example is the conceptual models to reason about Ownership Types in Rust by Crichton et al. [12]. The authors present two models: a dynamic and a static one. In our framework, each model can be seen as a notional machine. The authors divide each model into a formal model (about which formal statements can be made), which would correspond to the abstract representation of the notional machine, and an informal model, which would correspond to the concrete representation of the notional machine. In the dynamic model, our PL layer would be the Rust language and our NM layer would be Miri. In the static model, our PL layer would be the polonius model of borrow checking and our NM layer would be the permissions model of borrow checking, introduced by the authors.

6 Related Work

The term notional machine was coined by Du Boulay [16] to refer to “the idealised model of the computer implied by the constructs of the programming language”. Therefore it restricts the use of notional machines as means to help understand the runtime behavior of a program or the dynamic semantics of a language. Fincher et al. [19], a working group that included du Boulay, presented a thorough literature review and discussion of the term notional machine and established a broader definition as “pedagogic devices to assist the understanding of some aspect of programs or programming”, which includes uses of notional machines as means to help understand the static semantics of a language. The “Expressions as Trees” notional machine, for example, is used for both. We adopt here this broader view of notional machine.

Another related and important term is conceptual model. Fincher et al. [19] states that a “notional machine is effectively a special kind of conceptual model.” That’s important because conceptual models, such as the Substitution Model and the Environment Model used in the SICP book (Structure and Interpretation of Computer Programs) [2], and SICPJS [1], for example, are well understood to be essential for reasoning about programs. Notional machines that are sound are indeed conceptual models. The concrete representation of the notional machine can be thought of as embodying the visualization of this conceptual model.

The idea of simulating or representing a program by means of another program is an old one, and was first studied in detail in the 1970s by Milner [27] and Hoare [22]. Many of our notional machines illustrate a reduction, stepping, or evaluation ‘aspect’ of a programming language. Wadler et al. [33] has a nice description of how to relate such reduction systems with simulation, lock-step simulation, and bisimulation. The commutative diagram describing the desired property of a notional machine appears in many places in the literature, and is a basic concept in Category Theory. Closer to our application is its use as ‘promotion condition’ [3, 34].

Where computing education researchers capture program behavior through notional machines, programming language researchers instead use semantics [25]. Our work can be seen as a rather standard approach to show the correctness of one kind of semantics of (part of) a programming language, most often the operational semantics, with respect to another semantics, often a reduction semantics. An example of such an approach has been described by Clements et al. [10], whose Elaboration Theorem describes a property that is very similar to our soundness requirement. The lack of a formal approach to showing the soundness of notional machines is also noted by Pollock et al. [29], who develop a formal approach to specifying correct program state visualization tools, based on an executable semantics of the programming language formulated in the K framework [30]. In this paper we study a broader collection of notional machines than just program state visualization tools, and we apply our approach to study the soundness of notional machines.

Computing educators practitioners use a diverse set of notional machines [19]. Some notional machines form the basis of automated tools. The BlueJ IDE, which features prominently in an introductory programming textbook [24], includes a graphical user interface to visualize objects, invoke methods, and inspect object state. PythonTutor [21], an embeddable web-based program visualization system, is used by hundreds of thousands of users to visualize the execution of code written in Python, Java, JavaScript, and other programming languages. UUhistle [32], a “visual program simulation” system, takes a different approach: instead of visualizing program executions, it requires students to perform the execution steps in a constrained interactive environment. When developing such widely used tools, starting from a sound notional machine is essential.

Dickson et al. [14] discuss the issues around developing and using a notional machine in class. They note, amongst others, “that a notional machine must by definition be correct, but a student’s mental model of the notional machine often is not”, and that “specifying a notional machine was more difficult than we thought it would be”. Our work can help in developing a notional machine, and pointing out flaws in it.

7 Conclusions

Notional machines are popular in computer science education, commonly used both by instructors in their teaching practice as well as by researchers. Despite their popularity, there has been no precise formal characterization of what should be the relationship between a notional machine and the aspect of the programming language under its focus that would allow one to evaluate whether or not they are consistent with each other. We, therefore, introduced a definition of soundness for notional machines. The definition is based on simulation, a well-established notion widely used in many areas of computer science. Demonstrating soundness essentially amounts to constructing a commutative diagram relating the notional machine with the object of its focus.

Using this definition, we showed how we can (1) systematically design notional machines that are sound by construction, and (2) analyze existing notional machines to uncover inconsistencies and suggest improvements.

An important insight in the process is to distinguish between the concrete representation of the notional machine (typically visual) and its abstract representation, about which we can make formal statements. This distinction is akin to the distinction between the concrete and the abstract syntaxes of a programming language.

We then evaluated how applicable our approach is to notional machines actually used in practice. Using a set of previously published notional machines used in practice, we characterize their design space and show that the notional machines we analyzed using our approach are representative of that design space.

This work intends more generally to establish a framework to reason about notional machines, placing the research on notional machines on firmer ground. As such, it can be used to address challenges such as the design, analysis, and evaluation of notional machines, and the construction of automated tools based on notional machines.

Data-Availability Statement

The artifact implementing the programming languages, notional machines, and the relationship between them as described in the paper is available at [10.5281/zenodo.12609636](https://doi.org/10.5281/zenodo.12609636). The implementation of some of the notional machines also includes a concrete representation.

References

- [1] Harold Abelson and Gerald Jay Sussman. 2022. *Structure and Interpretation of Computer Programs: JavaScript Edition*. MIT Press.

- [2] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. 1996. *Structure and Interpretation of Computer Programs*, (Second Edition) (second edition ed.). Vol. 33.
- [3] R. S. Bird. 1984. The Promotion and Accumulation Strategies in Transformational Programming. *ACM Transactions on Programming Languages and Systems* 6, 4 (Oct. 1984), 487–504. <https://doi.org/10.1145/1780.1781>
- [4] R. S. Bird. 1989. Algebraic Identities for Program Calculation. *Comput. J.* 32, 2 (April 1989), 122–126. <https://doi.org/10.1093/comjnl/32.2.122>
- [5] N.G. de Bruijn. 1972. Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (Jan. 1972), 381–392. [https://doi.org/10.1016/1385-7258\(72\)90034-0](https://doi.org/10.1016/1385-7258(72)90034-0)
- [6] Jerome Bruner. 1960. *The Process of Education*. Harvard University Press.
- [7] Luca Chiodini, Igor Moreno Santos, Andrea Gallidabino, Anya Tafiiovich, André L. Santos, and Matthias Hauswirth. 2021. A Curated Inventory of Programming Language Misconceptions. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '21)*. Association for Computing Machinery, New York, NY, USA, 380–386. <https://doi.org/10.1145/3430665.3456343>
- [8] Alonzo Church. 1936. An Unsolvable Problem of Elementary Number Theory. *American Journal of Mathematics* 58, 2 (1936), 345–363. <https://doi.org/10.2307/2371045> jstor:2371045
- [9] Alonzo Church. 1941. *The Calculi of Lambda-Conversion*. Number 6. Princeton University Press.
- [10] John Clements, Matthew Flatt, and Matthias Felleisen. 2001. Modeling an Algebraic Stepper. In *Proceedings of the 10th European Symposium on Programming Languages and Systems (ESOP '01)*. Springer-Verlag, Berlin, Heidelberg, 320–334.
- [11] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. <https://doi.org/10.1145/512950.512973>
- [12] Will Crichton, Gavin Gray, and Shriram Krishnamurthi. 2023. A Grounded Conceptual Model for Ownership Types in Rust. *Proceedings of the ACM on Programming Languages* 7, OOPSLA2 (Oct. 2023), 265:1224–265:1252. <https://doi.org/10.1145/3622841>
- [13] Andrew R. Dalton and William Krehling. 2010. Automated Construction of Memory Diagrams for Program Comprehension. In *Proceedings of the 48th Annual Southeast Regional Conference (ACM SE '10)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/1900008.1900040>
- [14] Paul E. Dickson, Tim Richards, and Brett A. Becker. 2022. Experiences Implementing and Utilizing a Notional Machine in the Classroom. In *Proceedings of the 53rd ACM Technical Symposium V.1 on Computer Science Education (SIGCSE 2022)*. Association for Computing Machinery, New York, NY, USA, 850–856. <https://doi.org/10.1145/3478431.3499320>
- [15] Toby Dragon and Paul E. Dickson. 2016. Memory Diagrams: A Consistent Approach Across Concepts and Languages. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education (SIGCSE '16)*. Association for Computing Machinery, New York, NY, USA, 546–551. <https://doi.org/10.1145/2839509.2844607>
- [16] Benedict Du Boulay. 1986. Some Difficulties of Learning to Program. *Journal of Educational Computing Research* 2, 1 (Feb. 1986), 57–73. <https://doi.org/10.2190/3LFX-9RRF-67T8-UVK9> arXiv:https://doi.org/10.2190/3LFX-9RRF-67T8-UVK9
- [17] Benedict Du Boulay and Tim O'Shea. 1976. *How to Work the LOGO Machine*. Technical Report 4. Department of Artificial Intelligence, University of Edinburgh.
- [18] Richard Feynman. 1985. *Surely You're Joking, Mr. Feynman!* W. W. Norton.
- [19] Sally Fincher, Johan Jeuring, Craig S. Miller, Peter Donaldson, Benedict du Boulay, Matthias Hauswirth, Arto Hellas, Felienne Hermans, Colleen Lewis, Andreas Mühling, Janice L. Pearce, and Andrew Petersen. 2020. Notional Machines in Computing Education: The Education of Attention. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '20)*. Association for Computing Machinery, New York, NY, USA, 21–50. <https://doi.org/10.1145/3437800.3439202>
- [20] Jeremy Gibbons. 2002. Calculating Functional Programs. In *Algebraic and Coalgebraic Methods in the Mathematics of Program Construction: International Summer School and Workshop Oxford, UK, April 10–14, 2000 Revised Lectures*, Roland Backhouse, Roy Crole, and Jeremy Gibbons (Eds.). Springer, Berlin, Heidelberg, 151–203. https://doi.org/10.1007/3-540-47797-7_5
- [21] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for Cs Education. In *Proceeding of the 44th ACM Technical Symposium on Computer Science Education - SIGCSE '13*. ACM Press, Denver, Colorado, USA, 579. <https://doi.org/10.1145/2445196.2445368>
- [22] C. A. Hoare. 1972. Proof of Correctness of Data Representations. *Acta Informatica* 1, 4 (Dec. 1972), 271–281. <https://doi.org/10.1007/BF00289507>
- [23] Mark A. Holliday and David Luginbuhl. 2004. CS1 Assessment Using Memory Diagrams. In *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '04)*. Association for Computing Machinery, New York, NY, USA, 200–204. <https://doi.org/10.1145/971300.971373>
- [24] Michael Kölling and David Barnes. 2017. *Objects First With Java: A Practical Introduction Using BlueJ* (6th ed.). Pearson.
- [25] Shriram Krishnamurthi and Kathi Fisler. 2019. Programming Paradigms and Beyond. In *The Cambridge Handbook of Computing Education Research*, Anthony V. Robins and Sally A. Fincher (Eds.). Cambridge University Press, Cambridge, 377–413. <https://doi.org/10.1017/9781108654555.014>
- [26] Guillaume Marceau, Kathi Fisler, and Shriram Krishnamurthi. 2011. Do Values Grow on Trees?: Expression Integrity in Functional Programming. In *Proceedings of the Seventh International Workshop on Computing Education Research - ICER '11*. ACM Press, Providence, Rhode Island, USA, 39. <https://doi.org/10.1145/2016911.2016921>
- [27] Robin Milner. 1971. An Algebraic Definition of Simulation between Programs. In *Proceedings of the 2nd International Joint Conference on Artificial Intelligence (IJCAI '71)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 481–489.
- [28] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press, Cambridge, Mass.

- [29] Josh Pollock, Jared Roesch, Doug Woos, and Zachary Tatlock. 2019. Theia: Automatically Generating Correct Program State Visualizations. In *Proceedings of the 2019 ACM SIGPLAN Symposium on SPLASH-E (SPLASH-E 2019)*. Association for Computing Machinery, New York, NY, USA, 46–56. <https://doi.org/10.1145/3358711.3361625>
- [30] Grigore Roşu and Traian Florin Şerbănuță. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (Aug. 2010), 397–434. <https://doi.org/10.1016/j.jlap.2010.03.012>
- [31] Juha Sorva, Jan Lönnberg, and Lauri Malmi. 2013. Students' Ways of Experiencing Visual Program Simulation. *Computer Science Education* 23, 3 (Sept. 2013), 207–238. <https://doi.org/10.1080/08993408.2013.807962>
- [32] Juha Sorva and Teemu Sirkä. 2010. UUhistle: A Software Tool for Visual Program Simulation. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research - Koli Calling '10*. ACM Press, Berlin, Germany, 49–54. <https://doi.org/10.1145/1930464.1930471>
- [33] Philip Wadler, Wen Kokke, and Jeremy G. Siek. 2020. *Programming Language Foundations in Agda*.
- [34] Meng Wang, Jeremy Gibbons, Kazutaka Matsuda, and Zhenjiang Hu. 2013. Refactoring Pattern Matching. *Science of Computer Programming* 78, 11 (Nov. 2013), 2216–2242. <https://doi.org/10.1016/j.scico.2012.07.014>
- [35] Ryan Yates and Brent A. Yorgey. 2015. Diagrams: A Functional EDSL for Vector Graphics. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Functional Art, Music, Modelling and Design (FARM 2015)*. Association for Computing Machinery, New York, NY, USA, 4–5. <https://doi.org/10.1145/2808083.2808085>
- [36] Brent A. Yorgey. 2012. Monoids: Theme and Variations (Functional Pearl). *ACM SIGPLAN Notices* 47, 12 (Sept. 2012), 105–116. <https://doi.org/10.1145/2430532.2364520>

A Programming Language Definitions

The programming languages used throughout the paper mostly follows the presentation in the book *Types and Programming Languages* by Pierce [28] with minor changes. In particular, italics are used for metavariables and the axioms in the reduction (evaluation) rules and typing rules are shown with explicitly empty premises.

A.1 UntypedLambda

Figure 21 shows the syntax and evaluation rules for the untyped lambda calculus by Church [8, 9], that we have referred to as `UNTYPEDLAMBDA`. This language is used in Sections 2.1, 2.2, and 3.1.

Syntax	Evaluation
$ \begin{array}{ll} t ::= & \text{terms:} \\ & x \quad \text{variable} \\ & \lambda x. t \quad \text{abstraction} \\ & t t \quad \text{application} \\ v ::= & \text{values:} \\ & \lambda x. t \end{array} $	$ \begin{array}{c} \frac{}{(\lambda x. t_{12}) v_2 \longrightarrow [x \mapsto v_2] t_{12}} \text{E-APPABS} \\ \frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{E-APP1} \\ \frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \text{E-APP2} \end{array} $

Fig. 21. The untyped lambda calculus (`UNTYPEDLAMBDA`).

A.2 TypedArith

We define here the language `TYPEDARITH`, used in Section 2.4. Figure 22 shows its syntax and evaluation (reduction) rules and Figure 23 shows its typing rules. The appeal of using this language to present a notional machine focused on the types is its simplicity. Terms don't require type annotations and the typing rules don't require a type environment. In fact, Pierce uses it as the simplest example of a typed language when introducing type safety.

A.3 TypedLambdaRef

In Section 2.3, we showed the language `TYPEDLAMBDAREF`, used to design a notional machine that focuses on references. This language is composed of the simply-typed lambda calculus, the `TYPEDARITH` language, tuples, the `Unit` type, sequencing, and references. Our goal is again simplicity and this is the simplest language we need for the examples in the book that use the diagram. Figure 24 shows its syntax and evaluation (reduction) rules. We show only the reduction rules for sequencing, references, and tuples because the rules for the rest of the language are similar to what we showed before, except for the store then needs to be threaded through all the rules. Although that is a typed language, we don't present its typing rules because the notional machine in Section 2.3 is focused only on its runtime behavior and not its types.

Syntax

$t ::=$	terms:
true	constant true
false	constant false
if t then t else t	conditional
0	constant zero
succ t	successor
pred t	predecessor
iszero t	zero test
$v ::=$	values:
true	
false	
nv	
$nv ::=$	numeric values:
0	
succ nv	

Evaluation

$\frac{}{\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2}$	E-IfTRUE
$\frac{}{\text{if false then } t_2 \text{ else } t_3 \rightarrow t_3}$	E-IfFALSE
$\frac{t_1 \rightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3}$	E-If
$\frac{t_1 \rightarrow t'_1}{\text{succ } t_1 \rightarrow \text{succ } t'_1}$	E-SUCC
$\frac{}{\text{pred } 0 \rightarrow 0}$	E-PREDZERO
$\frac{}{\text{pred } (\text{succ } nv_1) \rightarrow nv_1}$	E-PREDSUCC
$\frac{t_1 \rightarrow t'_1}{\text{pred } t_1 \rightarrow \text{pred } t'_1}$	E-PRED
$\frac{}{\text{iszero } 0 \rightarrow \text{true}}$	E-ISZEROZERO
$\frac{}{\text{iszero } (\text{succ } nv_1) \rightarrow \text{false}}$	E-ISZEROSUCC
$\frac{t_1 \rightarrow t'_1}{\text{iszero } t_1 \rightarrow \text{iszero } t'_1}$	E-ISZERO

Fig. 22. Syntax and reduction rules of the TYPEDARITH language.

Syntax

$T ::=$	types:
Bool	type of booleans
Nat	type of natural numbers

Typing rules

$\frac{}{\text{true} : \text{Bool}}$	T-TRUE
$\frac{}{\text{false} : \text{Bool}}$	T-FALSE
$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$	T-If
$\frac{}{0 : \text{Nat}}$	T-ZERO
$\frac{t_1 : \text{Nat}}{\text{succ } t_1 : \text{Nat}}$	T-SUCC
$\frac{t_1 : \text{Nat}}{\text{pred } t_1 : \text{Nat}}$	T-PRED
$\frac{t_1 : \text{Nat}}{\text{iszero } t_1 : \text{Bool}}$	T-ISZERO

Fig. 23. Syntax of types and typing rules of the TYPEDARITH language.

Syntax			Evaluation		
1509	Syntax				
1510	$t ::=$	terms:			
1511	x	variable			
1512	$\lambda x : T. t$	abstraction			
1513	$t \ t$	application			
1514	true	boolean true			
1515	false	boolean false			
1516	0	zero			
1517	$\text{succ } t$	successor			
1518	$\text{pred } t$	predecessor			
1519	$\text{iszero } t$	zero test			
1520	unit	unit constant			
1521	$t; t$	sequence			
1522	$\text{ref } t$	reference creation			
1523	$!t$	dereference			
1524	$t := t$	assignment			
1525	l	location			
1526	$\{t_i^{i \in 1..n}\}$	tuple			
1527	$t.i$	projection			
1528	$v ::=$	values:			
1529	$\lambda x : T. t$				
1530	true				
1531	false				
1532	0				
1533	$\text{succ } v$				
1534	unit				
1535	l				
1536	$\{v_i^{i \in 1..n}\}$				
1537	$T ::=$	types:			
1538	$T \rightarrow T$	function type			
1539	Bool	boolean type			
1540	Nat	natural number type			
1541	Unit	unit type			
1542	$\text{Ref } T$	reference type			
1543	$\{T_i^{i \in 1..n}\}$	tuple type			
1544	$\mu ::=$	store:			
1545	$\emptyset$	empty store			
1546	$\mu, l \mapsto v$	location binding			
1547					
1548					
1549					
1550					
1551					
1552					
1553					
1554					
1555					
1556					
1557					
1558					
1559					
1560					

Fig. 24. TYPEDLAMBDAREF: Syntax and Evaluation